

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ ЛЬВІВСЬКИЙ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА ФАКУЛЬТЕТ
УПРАВЛІННЯ ФІНАНСАМИ ТА БІЗНЕСУ**

Кафедра цифрової економіки та бізнес-аналітики

КУРСОВА РОБОТА

з моделювання та автоматизації бізнес-процесів

на тему:

Інформаційна система ресторану

Спеціальність: 051 «Економіка»

Спеціалізація: «Інформаційні технології в бізнесі»

Освітній ступінь: бакалавр

Науковий керівник:

к.е.н., доцент Ярема О.Р.

(науковий ступінь, посада, прізвище, ініціали)

_____ “ _____ ” _____ 2026 р.
(підпис)

Виконавець:

Студентка групи УФЕ-31с

Дідух Т.А.
(прізвище, ініціали)

_____ “ _____ ” _____ 2026 р.
(підпис)

Загальна кількість балів _____

(підпис, ППП члена комісії)

(підпис, ППП члена комісії)

(підпис, ППП члена комісії)

ЛЬВІВ 2026

ЗМІСТ

ВСТУП.....	3
РОЗДІЛ 1. АНАЛІЗ ВИМОГ ДЛЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ БІЗНЕС- ПРОЦЕСІВ РЕСТОРАНУ	5
1.1. Постановка завдання.....	5
1.2. Розробка моделі варіантів використання вебсайту.....	7
1.3. Аналіз засобів реалізації	9
РОЗДІЛ 2. РОЗРОБКА БАЗИ ДАНИХ РЕСТОРАНУ	11
2.1. Опис моделі даних	11
2.2. Нормалізація відношень.....	15
2.3. Визначення типів даних	17
2.4. Обмеження цілісності даних	20
2.5. Реалізація SQL-скрипту	22
РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ.....	24
3.1. Вибір середовища розробки та технологічного стеку.....	24
3.2. Структура вебсайту.....	26
3.3. Макети сторінок веб-сайту	29
3.4. Програмування серверної частини.....	33
3.5. Програмування клієнтської частини.....	37
3.6. Розміщення веб-сайту на локальному віртуальному середовищі або в Інтернеті.....	39
3.7. Тестування взаємодії вебдодатка з базою даних	41
ВИСНОВОК	44
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	46

ВСТУП

Автоматизація бізнес-процесів закладів ресторанного господарства має суттєві відмінності від універсальної комп'ютерної автоматизації підприємств, де застосовуються стандартні CRM та ERP-системи. Уніфікація робочих процесів у сфері громадського харчування дозволяє створювати спеціалізовані інформаційні системи, які ефективно інтегрують внутрішній облік та зовнішню взаємодію з клієнтами [1].

Повноцінний вебсайт ресторану виконує не лише інформаційну, а й комерційну функцію, оскільки містить актуальні дані про страви, ціни та забезпечує можливість дистанційного оформлення замовлень. Належна організація документообігу та обліку страв, напоїв, а також контроль діяльності персоналу є критично важливими етапами у вдосконаленні діяльності та підвищенні конкурентоспроможності закладу [3].

Мета дослідження полягає у проектуванні та розробці інформаційної системи для автоматизації бізнес-процесів ресторану, яка забезпечить взаємодію з клієнтами через вебдодаток та ефективне управління замовленнями для персоналу.

Завдання дослідження:

1. Проаналізувати предметну область та розробити модель варіантів використання системи.
2. Спроекувати реляційну базу даних, здійснити її нормалізацію та визначити обмеження цілісності.
3. Розробити серверну та клієнтську частини вебдодатка з реалізацією функцій авторизації та розмежування прав доступу.
4. Провести тестування розробленого програмного забезпечення.

Об'єкт дослідження - процеси обслуговування клієнтів та обробки замовлень у закладі ресторанного господарства.

Предмет дослідження - методи та засоби проектування баз даних і розробки веборієнтованих інформаційних систем для ресторанного бізнесу.

Практичне значення отриманих результатів. Розроблена система оптимізує роботу персоналу, скорочує час на обробку замовлень та бронювання столиків, а також підвищує ефективність управління закладом. Програмний продукт адаптований для впровадження у діяльність реального закладу харчування та дозволяє автоматизувати цикл від прийому замовлення до його обліку [3].

Використане програмне забезпечення. Проектування бази даних здійснювалося за допомогою MySQL Workbench. Клієнтська частина реалізована з використанням HTML, CSS та JavaScript. Серверна частина розроблена мовою PHP з використанням СКБД MySQL.

Структура роботи. Курсова робота складається зі вступу, трьох розділів основної частини, висновків, списку використаних джерел.

РОЗДІЛ 1. АНАЛІЗ ВИМОГ ДЛЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ БІЗНЕС-ПРОЦЕСІВ РЕСТОРАНУ

1.1. Постановка завдання

Організація роботи закладу харчування потребує чіткої координації між залом, кухнею та адміністрацією. Впровадження спеціалізованої інформаційної системи дозволяє автоматизувати ключові процеси: від прийому замовлень до контролю їх виконання, розподілу навантаження на персонал та ведення клієнтської бази [3].

Для виконання цих завдань проектується веборієнтована інформаційна система ресторану, яка об'єднує публічний клієнтський сервіс та закритий робочий термінал для персоналу. Архітектура системи побудована на єдиній точці авторизації з автоматичним розмежуванням прав доступу на основі реляційної бази даних. Система обслуговує чотири цільові категорії користувачів: клієнтів, адміністраторів, кухарів та офіціантів.

Клієнтська частина системи забезпечує такий функціонал:

- Перегляд інтерактивного меню закладу з актуальними цінами.
- Реєстрація та авторизація користувачів для безпечного доступу до особистого кабінету.
- Багатокомпонентне оформлення замовлень на доставку (вибір страв, розрахунок фінальної вартості, фіксація способу оплати та адреси).
- Бронювання столиків на вказану дату та час із передачею контактних даних.
- Участь в автоматизованій програмі лояльності (нарахування 10% кешбеку з кожного замовлення на бонусний рахунок).

- Публікація відгуків на сайті (з автоматичною ідентифікацією авторизованих користувачів або опцією анонімного коментування).

Адміністративна частина (робочий термінал) розділена на функціональні модулі відповідно до посад:

- Для адміністратора: моніторинг активних замовлень на доставку, контроль фінансової інформації (сума, тип оплати) та зміна статусу замовлення на «Виконано» після його фактичної передачі клієнту.

- Для кухаря: доступ до ізольованого кухонного інтерфейсу, який генерує перелік страв та їх кількість для поточних чеків, виключаючи доступ до фінансових або персональних даних замовника.

- Для офіціанта: перегляд списку актуальних бронювань залу з відображенням кількості гостей, часу резервації та клікабельними номерами телефонів для швидкого зв'язку.

Такий підхід до проєктування гарантує цілісність даних, підвищує швидкість обслуговування та мінімізує інформаційне перевантаження працівників.

1.2. Розробка моделі варіантів використання вебсайту

Модель варіантів використання (Use Case) визначає функціональні вимоги до інформаційної системи ресторану та демонструє сценарії взаємодії користувачів (акторів) із програмним продуктом. Розроблена діаграма деталізує бізнес-процеси закладу та виділяє чотирьох окремих акторів: «Клієнт», «Адміністратор», «Кухар» та «Офіціант». Це дозволяє точно розмежувати права доступу та функціональні обов'язки в системі.

Базовим прецедентом для всіх без винятку користувачів є «Авторизація в системі». Цей процес забезпечує єдину точку входу та автоматично розподіляє робочі інтерфейси чи відкриває клієнтський кабінет відповідно до статусу, збереженого в базі даних.

Актор «Клієнт» ініціює процеси отримання послуг. До його базових прецедентів належать: перегляд меню, можливість залишити відгук, перегляд стану бонусного рахунку, забронювати столик та оформити доставку. Процес оформлення доставки має обов'язкову залежність (зв'язок <<включає>>) від прецеденту «Оплатити замовлення», оскільки фіксація методу розрахунку є невід'ємним етапом формування чека.

Внутрішні операційні процеси ресторану чітко розподілені між трьома працівниками. Актор «Адміністратор» виконує контролюючі функції: використовує прецеденти «Переглянути замовлення на доставку» та «Змінити статус замовлення» для моніторингу активних чеків і переведення їх у статус виконаних, також має право доступу до модуля аналітичної звітності, що дозволяє формувати звіти про доходи за певний період, відстежувати популярність страв та контролювати ефективність роботи персоналу.

Актор «Кухар» працює з ізольованим кухонним модулем через прецедент «Переглянути страви для приготування», отримуючи автоматично згенерований список позицій без доступу до фінансової інформації. Актор «Офіціант» відповідає

за координацію гостей у залі, застосовуючи прецедент «Переглянути список бронювань» для доступу до резервів столиків.

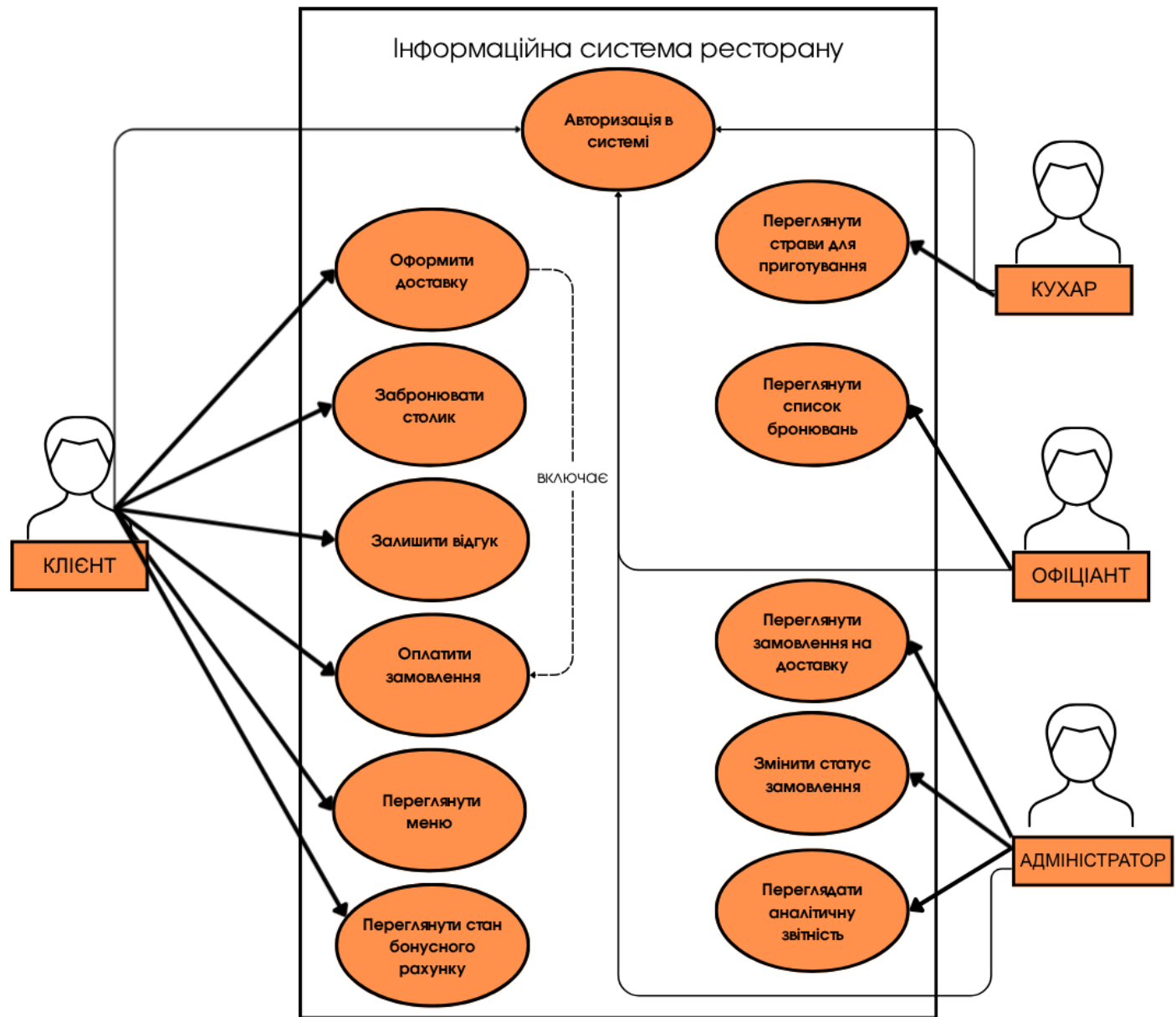


Рис. 1.1. Use Case Diagram інформаційної системи ресторану

Джерело: розроблено автором

1.3. Аналіз засобів реалізації

Для розробки веборієнтованої інформаційної системи ресторану обрано стек технологій, що забезпечує стабільну роботу клієнтської та серверної частин, а також зручність адміністрування бази даних. Вибір інструментів базується на вимогах до продуктивності, безпеки та кросплатформності вебдодатка.

Засоби розробки клієнтської частини (Frontend).

Клієнтська частина реалізована за допомогою базових вебтехнологій для забезпечення максимальної сумісності з сучасними браузерами:

- HTML5 (HyperText Markup Language) використовується для побудови семантичної структури вебсторінок. За допомогою HTML-форм реалізовано інтерфейси реєстрації, авторизації, бронювання столиків та публікації відгуків.

- CSS3 (Cascading Style Sheets) застосовується для візуального оформлення системи та забезпечення адаптивності інтерфейсу (через @media-запити). Використані технології Flexbox та CSS Grid для позиціювання елементів меню та карток відгуків. Для типографіки підключено шрифти Google Fonts (Playfair Display для заголовків та Montserrat для основного тексту).

- JavaScript забезпечує інтерактивність на стороні клієнта. Використовується для реалізації логіки кошика: додавання страв, зміна кількості порцій та динамічний перерахунок фінальної вартості замовлення без перезавантаження сторінки.

Засоби розробки серверної частини (Backend) та бази даних.

- PHP (Hypertext Preprocessor) виконує роль основної серверної мови програмування. Забезпечує обробку HTTP-запитів, роботу із сесіями (session_start()), маршрутизацію користувачів залежно від їхньої ролі (адміністратор, кухар, офіціант, клієнт) та взаємодію з базою даних через

розширення `mysqli`. Також PHP використовується для криптографічного захисту даних (функції `password_hash` та `password_verify`).

- MySQL - реляційна система управління базами даних. Відповідає за зберігання та обробку структурованої інформації. Забезпечує реляційну цілісність даних через використання зовнішніх ключів (Foreign Keys) та строгих типів даних (наприклад, ENUM для визначення посад персоналу).

Програмне середовище та інструментарій розробника.

Проектування, написання коду та тестування системи відбувалося з використанням спеціалізованого програмного забезпечення:

- MAMP (Mac, Apache, MySQL, PHP) - це локальне серверне середовище для операційної системи macOS. Забезпечує розгортання вебсервера Apache та локального сервера баз даних MySQL, що дозволяє тестувати систему автономно без використання віддаленого хостингу.

- Visual Studio Code - основне середовище розробки (IDE) для написання та форматування програмного коду (HTML, CSS, PHP). Обрано завдяки високій швидкості роботи та вбудованим інструментам підсвічування синтаксису.

- MySQL Workbench - це професійний графічний інструмент для візуального проектування та адміністрування баз даних. Використовувався для створення таблиць, написання SQL-запитів, контролю за виконанням транзакцій (Action Output) та ручного коригування записів персоналу.

- Safari: веббраузер, який застосовувався для тестування користувацького досвіду, перевірки адаптивності інтерфейсу та налагодження клієнтського коду.

РОЗДІЛ 2. РОЗРОБКА БАЗИ ДАНИХ РЕСТОРАНУ

2.1. Опис моделі даних

Аналіз предметної області складається з аналізу даних та аналізу завдань. До ключового результату етапу аналізу належить узгоджена діаграма сутностей і зв'язків. Процес моделювання полягає у виділенні об'єктів (сутностей предметної області), визначенні їхніх властивостей і виявленні зв'язків між ними. Розроблена концептуальна модель бази даних ресторану складається з дев'яти взаємопов'язаних сутностей, кожна з яких має визначений набір атрибутів для класифікації та відображення їхнього стану.

Концептуальна модель бази даних розроблена з урахуванням реляційної архітектури та складається з дев'яти взаємопов'язаних сутностей. Структура забезпечує зберігання інформації про клієнтів, персонал, меню, фінансові операції та процеси бронювання, виключаючи дублювання ключових даних.

1. Сутність Users (Користувачі) - центральний елемент системи, що зберігає облікові записи.

- Атрибути: унікальний ідентифікатор (`user_id`), ім'я та прізвище, електронна пошта, контактний телефон, криптографічний хеш пароля. Для забезпечення функцій доставки та роботи програми лояльності виділено атрибути адреси (місто, вулиця, будинок, квартира) та баланс бонусів (`loyalty_points`).

- Зв'язки: виступає головною сутністю для замовлень, оплат, бронювань та відгуків (відношення «один-до-багатьох»).

2. Сутність Employees (Персонал) - реєстр працівників закладу.

- Атрибути: ідентифікатор працівника, ім'я та прізвище, електронна пошта, контактний телефон, криптографічний хеш пароля, посада (`position`)

- Зв'язки: працівник обробляє замовлення (Orders) та адмініструє бронювання (Reservations).

3. Сутність Categories (Категорії меню) - класифікатор ресторанного меню.

- Атрибути: ідентифікатор категорії, назва, текстовий опис, посилання на зображення, статус активності.

- Зв'язки: пов'язана відношенням «один-до-багатьох» із сутністю Dishes.

4. Сутність Dishes (Страви) - детальний каталог позицій.

- Атрибути: ідентифікатор страви, ідентифікатор категорії, назва, опис, вартість, вага порції у грамах, маркер доступності для замовлення.

- Зв'язки: кожна страва може фігурувати у багатьох замовленнях через проміжну таблицю деталей.

5. Сутність Orders (Замовлення) - фіксує операції з доставки їжі.

- Атрибути: номер замовлення, дата і час оформлення, загальна сума чека, поточний статус виконання.

- Зв'язки: містить зовнішній ключ employee_id (ідентифікація відповідального працівника). Зв'язок із клієнтом реалізовано суворо через таблицю Payments (відношення 1:1).

6. Сутність Order_details (Деталі замовлення) - розшифровує склад конкретного кошика.

- Атрибути: унікальний ідентифікатор запису, кількість замовлених порцій, ціна за одиницю, проміжна сума.

- Зв'язки: вирішує реляційний зв'язок «багато-до-багатьох» між замовленнями (Orders) та стравами (Dishes).

7. Сутність Payments (Оплати) - облік фінансових транзакцій.

- Атрибути: ідентифікатор платежу, дата транзакції, сума, метод розрахунку, статус оплати. Для збереження точної історії доставки на момент розрахунку сутність містить власні поля адреси (місто, вулиця, будинок, квартира).

- Зв'язки: прив'язана до конкретного замовлення (`order_id`) та клієнта (`user_id`).

8. Сутність Reservations (Бронювання) - управління посадкою в залі.

- Атрибути: ідентифікатор броні, дата та час візиту, кількість осіб, номер виділеного столика, статус броні, особливі побажання, а також контактні дані гостя (ім'я та телефон) для ідентифікації резерву.

- Зв'язки: закріплюється за клієнтом (`user_id`) та працівником (`employee_id`), який підтвердив резерв.

9. Сутність reviews_ (Відгуки) - акумулює зворотний зв'язок від клієнтів.

- Атрибути: ідентифікатор відгуку, текстовий вміст коментаря, числова оцінка (`rating`), мітка часу публікації, ім'я автора.

- Зв'язки: пов'язана з профілем клієнта (`user_id`), що дозволяє системі авторизувати коментарі та відрізняти їх від записів анонімних користувачів.



Рис.2. ER-діаграма інформаційної системи ресторану

Джерело: розроблено автором

Представлена концептуальна ER-діаграма візуалізує бізнес-логіку взаємодії всіх компонентів системи. Вона підтверджує, що архітектура бази даних позбавлена ізольованих елементів: кожна сутність інтегрована у загальний процес обслуговування (від реєстрації клієнта до фіксації оплати). Таке попереднє структурування унеможливорює виникнення логічних конфліктів на етапі написання коду та гарантує, що база даних здатна обробити будь-який сценарій роботи ресторану.

2.2. Нормалізація відношень

Нормалізація бази даних - це метод проектування реляційної моделі, який передбачає декомпозицію таблиць для усунення надмірності даних та запобігання аномаліям оновлення, включення і видалення. Процес зводиться до послідовного приведення структури до нормальних форм, кожна з яких висуває жорсткіші вимоги до організації інформації. Розроблена модель бази даних ресторану пройшла перевірку на відповідність першій, другій та третій нормальним формам (1НФ, 2НФ, 3НФ) [6].

Перша нормальна форма (1НФ) вимагає, щоб усі атрибути відношення були атомарними (містили неподільні значення), а таблиця не мала повторюваних груп. У спроектованій базі даних усі поля відповідають цьому критерію. Наприклад, у таблиці Users інформація про клієнта розділена на окремі атрибути first_name та last_name, замість використання єдиного поля для повного імені. За таким же принципом адреса розбита на атомарні складові (city, street, house_number, apartment_number), що дозволяє коректно сортувати дані для доставки. Кожен запис має лише один номер телефону та електронну пошту. Для зберігання множинних значень (наприклад, кількох різних страв у межах одного замовлення) спроектовано окрему таблицю Order_details, що унеможливорює створення масивів даних всередині однієї комірки [6].

Друга нормальна форма (2НФ) передбачає, що відношення знаходиться у 1НФ, і кожен неключовий атрибут функціонально повно залежить від первинного ключа. Оскільки у розроблених таблицях (Users, Employees, Categories, Dishes, Orders, Reservations, Payments, reviews_) використовуються прості сурогатні первинні ключі з автоінкрементом, часткова залежність неключових атрибутів від ключа неможлива концептуально. У проміжній таблиці Order_details неключові атрибути quantity (кількість) та unit_price (ціна за одиницю) залежать безпосередньо

від факту додавання конкретної страви до конкретного чека, що гарантує виконання правил 2НФ [6].

Третя нормальна форма (3НФ) вимагає відсутності транзитивних залежностей між неключовими атрибутами. Ця вимога в нашій базі даних повністю реалізована через ізоляцію інформації у відповідні довідники. Наприклад, назва та опис категорії страв винесені в таблицю Categories, а таблиця Dishes містить лише зовнішній ключ category_id. Для забезпечення повної ізоляції сутностей, клієнти (Users) та персонал (Employees) розділені на дві незалежні таблиці. Це гарантує відсутність перетину логіки авторизації та відповідає вимогам нормалізації для чіткого розмежування бізнес-ролей.[6].

2.3. Визначення типів даних

Вибір типів даних визначає ефективність зберігання інформації та швидкість обробки запитів сервером бази даних. Для кожного атрибута реляційної моделі інформаційної системи ресторану підбрано оптимальний формат, що відповідає характеру реальних даних та вимогам до їх обробки:

- Цілочисельний тип (INT) використовується для первинних і зовнішніх ключів (user_id, order_id, dish_id тощо) із параметром AUTO_INCREMENT для автоматичної генерації унікальних ідентифікаторів системою. Також застосовується для кількісних показників: кількості замовлених порцій, кількості гостей при бронюванні, номеру столика та балансу накопиченого кешбеку в програмі лояльності.

- Рядковий тип змінної довжини (VARCHAR) призначений для текстової інформації з передбачуваним максимальним розміром. Формат VARCHAR(255) застосовано для імен, електронних адрес, назв страв та вулиць. Формат VARCHAR(100) використовується для назв міст, а VARCHAR(50) - для категорій і методів оплати. Формат VARCHAR(20) виділено спеціально для номерів телефонів, а VARCHAR(10) - для коротких значень номерів будинків та квартир.

- Текстовий тип (TEXT) застосовується для полів, довжину яких складно обмежити жорсткими рамками. У розробленій базі це розгорнуті описи страв у меню та безпосередній зміст коментарів від клієнтів у таблиці відгуків.

- Перелічуваний тип (ENUM) застосовується для жорсткої фіксації списку дозволених значень, що захищає базу даних від введення некоректної інформації. Цей тип забезпечує визначення посад персоналу (position у таблиці працівників) та поточних станів виконання (status у замовленнях та бронюваннях).

- Тип із фіксованою точкою (DECIMAL) обраний для всіх фінансових атрибутів: вартості окремої страви, загальної суми замовлення та розміру транзакції. Формат DECIMAL(10,2) гарантує точність обчислень до двох знаків після коми і запобігає фінансовим похибкам округлення, які виникають при використанні типів із плаваючою точкою.

- Дата і час (DATETIME) фіксує точний хронометраж подій у системі. Використовується для реєстрації часу створення замовлення (order_date), здійснення оплати (payment_date) та запланованого часу візиту гостей (reservation_date).

- Логічний тип (TINYINT) виконує роль булевої змінної (зі значеннями 0 або 1) для відображення статусів видимості. Поле is_available у таблиці страв позначає їх фізичну наявність для замовлення, а is_active контролює відображення категорій меню на клієнтській частині сайту.

- Часовий тип (TIMESTAMP) застосовано для поля created_at у таблиці reviews із параметром DEFAULT CURRENT_TIMESTAMP. Це забезпечує автоматичну фіксацію серверного часу в момент публікації нового відгуку без додаткових програмних запитів.

Фізична модель розробленої бази даних із зазначенням усіх таблиць, реляційних зв'язків та визначених типів даних, згенерована у середовищі MySQL Workbench, представлена на рис. 2.1. Ця фізична модель демонструє структуру бази даних, повністю адаптовану під вимоги СУБД MySQL.

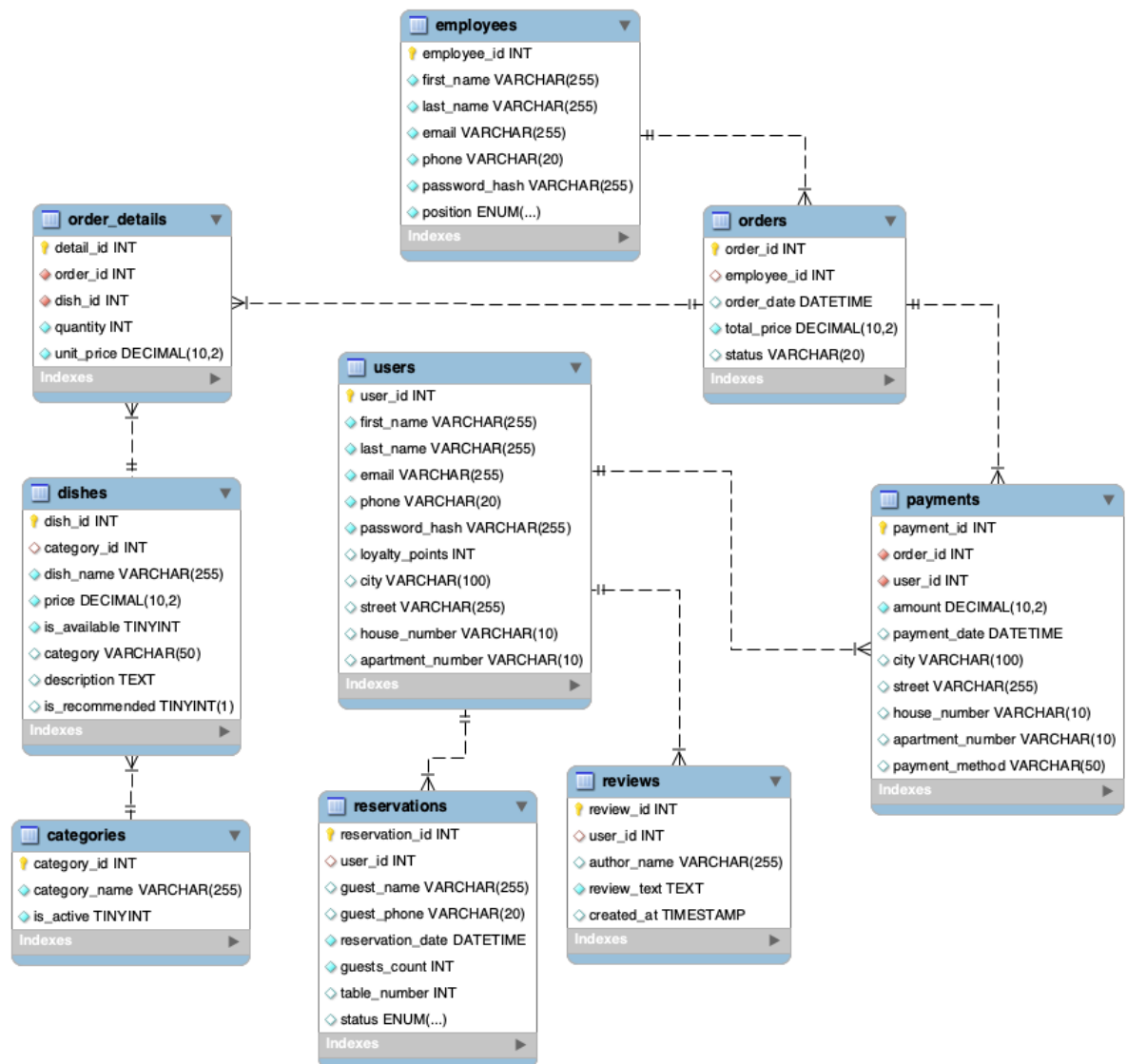


Рис 2.1 Фізична модель бази даних у середовищі MySQL

Джерело: розроблено автором

Дана фізична модель демонструє остаточну структуру бази даних, адаптовану під вимоги СУБД MySQL, тут жорстко зафіксовані типи даних для кожного атрибута та визначені механізми зовнішніх ключів. Ця модель є точним відображенням того, як інформація фізично зберігається на сервері. Вона слугує готовим технічним фундаментом для розробки серверної частини (backend) вебдодатка та написання SQL-запитів.

2.4. Обмеження цілісності даних

Обмеження цілісності - це набір правил, що гарантують коректність, несуперечливість та достовірність інформації у базі даних під час її внесення, оновлення чи видалення. У спроектованій реляційній моделі інформаційної системи ресторану реалізовано три основні типи обмежень: цілісність сутностей, цілісність посилань та цілісність домену.

Цілісність сутностей забезпечується визначенням первинних ключів (PRIMARY KEY) для кожної з дев'яти таблиць. Атрибути, такі як `user_id` у таблиці `Users` або `order_id` у таблиці `Orders`, є унікальними ідентифікаторами записів і не можуть містити порожніх значень. Це унеможливорює появу дубльованих або неідентифікованих рядків у базі даних.

Цілісність посилань (референційна цілісність) підтримується через механізм зовнішніх ключів (FOREIGN KEY). Наприклад, таблиця `Orders` містить зовнішній ключ `employee_id`. Це обмеження гарантує, що неможливо оформити замовлення для неіснуючого клієнта або призначити відповідальним за нього неіснуючого працівника. У разі спроби видалення запису клієнта з таблиці `Users`, якщо у нього існують пов'язані замовлення в таблиці `Orders` або транзакції у `Payments`, система керування базами даних заблокує цю дію (правило RESTRICT), що є необхідним для збереження фінансової та статистичної історії закладу.

Цілісність домену визначає допустимі формати та набори значень для конкретних атрибутів. У розробленій базі даних ресторану це реалізовано за допомогою таких інструментів:

- Обмеження типу даних ENUM застосовано для полів `position` (у `Employees`) та `status` (у замовленнях і бронюваннях). Воно фізично блокує запис у базу будь-яких значень, окрім чітко визначеного списку (наприклад, дозволяє лише 'manager', 'cook', 'waiter' для посад).

- Обмеження NOT NULL застосовано до критичних полів (наприклад, first_name, phone, total_price), що забороняє збереження запису в таблиці без наявності цієї базової інформації.

- Обмеження UNIQUE встановлено для поля email у таблиці Users. Це блокує можливість реєстрації кількох облікових записів на одну електронну адресу, запобігаючи дублюванню профайлів.

- Обмеження DEFAULT використано для автоматичного присвоєння початкових значень при створенні запису. Зокрема, новій створеній категорії в Categories надається логічний прапорець активності 1, а в таблиці reviews_ поле created_at автоматично фіксує поточний серверний час.

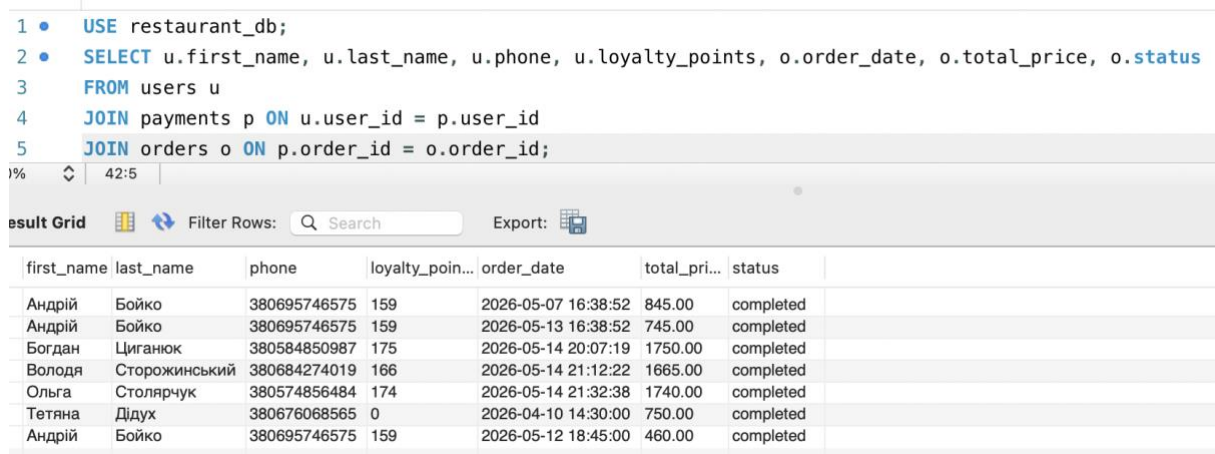
Впровадження описаних правил безпосередньо на рівні структури бази даних дозволяє делегувати відповідальність за перевірку інформації серверу MySQL. Це знижує ймовірність програмних помилок на рівні серверної частини вебдодатка (PHP) та підвищує загальну відмовостійкість інформаційної системи [6].

Свідомим архітектурним рішенням у системі є дозвіл на використання значень NULL для зовнішнього ключа user_id у таблиці reviews_. Це забезпечує гнучкість системи: дозволяє акумулювати відгуки як від авторизованих клієнтів (із жорсткою реляційною прив'язкою до профілю), так і від неавторизованих гостей (анонімно), не порушуючи при цьому загальних правил цілісності.

2.5. Реалізація SQL-скрипту

SQL-скрипт являє собою набір команд для створення таблиць, налаштування зв'язків та маніпуляції даними. Для фізичної реалізації бази даних ресторану використано середовище MySQL Workbench. Повний текст скрипту створення структури винесено у Додаток А.

Для перевірки працездатності системи розроблено тестові SQL-запити, які задовольняють основні бізнес-вимоги закладу. Для виведення зведеної інформації про клієнтів та їхні замовлення застосовано оператор JOIN. Він об'єднує записи з таблиць users та orders за спільним полем user_id. Це дозволяє отримати деталі замовлення разом із контактними даними гостя та його балансом лояльності в одній таблиці. (рис. 2.2).



```
1 • USE restaurant_db;
2 • SELECT u.first_name, u.last_name, u.phone, u.loyalty_points, o.order_date, o.total_price, o.status
3 FROM users u
4 JOIN payments p ON u.user_id = p.user_id
5 JOIN orders o ON p.order_id = o.order_id;
```

first_name	last_name	phone	loyalty_poin...	order_date	total_pri...	status
Андрій	Бойко	380695746575	159	2026-05-07 16:38:52	845.00	completed
Андрій	Бойко	380695746575	159	2026-05-13 16:38:52	745.00	completed
Богдан	Циганюк	380584850987	175	2026-05-14 20:07:19	1750.00	completed
Володя	Сторожинський	380684274019	166	2026-05-14 21:12:22	1665.00	completed
Ольга	Столярчук	380574856484	174	2026-05-14 21:32:38	1740.00	completed
Тетяна	Дідух	380676068565	0	2026-04-10 14:30:00	750.00	completed
Андрій	Бойко	380695746575	159	2026-05-12 18:45:00	460.00	completed

Рис. 2.2. Інформація про клієнтів, їхні замовлення та бонусний баланс

Аналіз фінансових надходжень ресторану реалізовано через агрегатні функції. Команда EXTRACT(MONTH FROM p.payment_date) виділяє номер місяця з точної дати транзакції. Функція SUM(p.amount) підраховує загальну суму доходів. Для отримання коректних фінансових результатів таблиця оплат об'єднується із таблицею замовлень, де умова WHERE o.status = 'completed' відфільтровує лише успішно закриті чеки. Оператор GROUP BY групує підраховані суми за місяцями, а ORDER BY вибудовує їх у хронологічному порядку (рис. 2.3).

```

1 • USE restaurant_db;
2 SELECT EXTRACT(MONTH FROM p.payment_date) AS month, SUM(p.amount) AS total_income
3 FROM payments p
4 JOIN orders o ON p.order_id = o.order_id
5 WHERE o.status = 'completed'
6 GROUP BY EXTRACT(MONTH FROM p.payment_date)
7 ORDER BY month;

```

0% 16:7

Result Grid Filter Rows: Search Export:

month	total_inco...
4	750.00
5	7205.00

Рис. 2.3. Обчислення доходів ресторану за місяцями

Для відстеження популярності позицій меню створено запит, який використовує оператор JOIN для об'єднання даних з таблиць dishes та order_details. Функція SUM(od.quantity) підраховує загальну кількість проданих порцій для кожної конкретної страви. Оператор GROUP BY групує дані за назвою позиції, а ORDER BY total_sold DESC сортує отримані результати за спаданням. Це дозволяє сформувати наочний рейтинг продажів - від найбільш затребуваних страв у меню до найменш популярних (рис. 2.4).

```

1 • USE restaurant_db;
2 • SELECT d.dish_name, SUM(od.quantity) AS total_sold
3 FROM dishes d
4 JOIN order_details od ON d.dish_id = od.dish_id
5 GROUP BY d.dish_name
6 ORDER BY total_sold DESC;
7

```

1:7

Result Grid Filter Rows: Search Export:

dish_name	total_sold
Стейк Рібай (Prime)	4
Картопля фрі з пармезаном	4
Стейк Стріплойн	4
Стейк Ті-Боун	3
Лимонад Класичний	2
Спаржа на грилі	2
Вино Мерло (келих)	2
Овочі гриль	1
Свинячі ребра в медовій глазури	1
Філе-мінйон	1

Рис. 2.4. Аналіз популярності страв у меню

РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ

3.1. Вибір середовища розробки та технологічного стеку

Для реалізації інформаційної системи ресторану було обрано комбінований стек технологій, що забезпечує гнучкість розробки, високу швидкість завантаження сторінок та стабільність роботи серверної частини:

- Frontend: HTML5, CSS3 та JavaScript. Ця тріада технологій застосовується для створення адаптивного та інтуїтивно зрозумілого інтерфейсу користувача, що коректно відображається на будь-яких пристроях (смартфонах, планшетах, десктопах). Особлива увага приділена динамічним елементам (кошик замовлень, форми бронювання та відгуків), які обробляються на стороні клієнта за допомогою JavaScript для мінімізації зайвих перезавантажень сторінок.

- Backend: мова програмування PHP версії 8.3. Вибір зумовлений її високою продуктивністю, повною підтримкою об'єктно-орієнтованого програмування та наявністю вбудованих засобів для безпечної роботи з реляційними базами даних. Серверні скрипти відповідають за обробку вхідних POST-запитів від користувачів, валідацію форм, хешування паролів та формування відповідей.

- Database: СКБД MySQL, розгорнута на віртуальному сервері під управлінням операційної системи Ubuntu. MySQL забезпечує високу швидкість виконання операцій читання та запису даних, підтримує механізми транзакцій (ACID) для фінансових операцій у таблиці payments та гарантує референційну цілісність завдяки зовнішнім ключам.

- Infrastructure: для забезпечення локального середовища розробки на платформі macOS використано програмний комплекс MAMP (пакет, що включає

вебсервер Apache, інтерпретатор PHP та СКБД MySQL), який дозволяє повноцінно симулювати роботу віддаленого вебсервера. Зв'язок із базою даних здійснюється через внутрішню мережу Parallels Desktop за IP-адресою 10.211.55.4.

Для взаємодії серверних PHP-скриптів із базою даних restaurant_db обрано розширення PDO (PHP Data Objects). На відміну від застарілих бібліотек, PDO забезпечує вищий рівень безпеки завдяки використанню підготовлених запитів (Prepared Statements), що повністю нівелює ризик проведення зловмисниками атак типу SQL-ін'єкцій. Крім того, архітектурні особливості PDO дозволяють за необхідності легко перейти на іншу СКБД (наприклад, PostgreSQL) без значної перебудови серверного коду додатка.

3.2. Структура вебсайту

Архітектура вебсайту ресторанного комплексу побудована за модульним принципом, що забезпечує розподіл функціональних можливостей для різних категорій користувачів: неавторизованих гостей, постійних клієнтів та адміністративного персоналу закладу (адміністраторів, кухарів, офіціантів). Взаємодія між сторінками реалізована через єдину точку входу, де маршрутизація запитів контролюється безпосередньо серверними PHP-скриптами.

Логічна структура вебсайту складається з таких основних блоків та сторінок:

- Головна сторінка (index.php): репрезентативна частина інформаційної системи, яка містить інформацію про заклад, акційні пропозиції та блок із останніми відгуками відвідувачів, які динамічно завантажуються з таблиці reviews.
- Меню ресторану (menu.php): каталог, де всі страви розподілені за відповідними категоріями. Сторінка підтягує актуальні дані з таблиць categories та dishes. Відображення страв контролюється логічним прапорцем is_available, що унеможливорює замовлення позицій, яких немає в наявності. Кожна страва оснащена кнопкою інтерактивного додавання до кошика.
- Кошик та оформлення доставки (cart.php, checkout.php): інтерфейс для перегляду обраних страв, редагування їхньої кількості та фінального підтвердження замовлення. При переході до оплати дані послідовно надходять у таблиці orders та order_details, а після введення адреси та фіксації транзакції створюється запис у таблиці payments.
- Сторінка онлайн-бронювання (reservation.php): форма для резервування столика із зазначенням дати, часу, кількості гостей та номера столика. Авторизовані користувачі оформлюють бронь через свій профіль, а гості сайту

можуть заповнити форму анонімно, вказавши ім'я та телефон. Система записує ці дані у таблицю `reservations`.

- Особистий кабінет клієнта (`profile.php`): захищений розділ сайту, доступний після авторизації. Користувач може бачити свій баланс у програмі лояльності (`loyalty_points`), який накопичується у вигляді бонусів з кожної оплати, а також переглядати історію своїх попередніх замовлень.

- Система авторизації та реєстрації (`login.php`, `register.php`): модулі, відповідальні за створення нових облікових записів та перевірку прав доступу. При реєстрації пароль користувача проходить обов'язкове криптографічне хешування перед збереженням у поле `password_hash` таблиці `users`.

- Адміністративна панель (`admin.php`): робочий простір для персоналу ресторану. Доступ до нього обмежується типом ролі користувача (`role` в `users`). Адміністратори мають доступ до керування замовленнями, бронюваннями та перегляду фінансової аналітики надходжень за місяцями. Кухарі та офіціанти бачать лише замовлення зі статусами `new` або `in progress` для оперативного виконання робочих процесів.

Взаємозв'язки між усіма компонентами системи та логіка переходів користувача відображені на карті сайту (рис. 3.1).

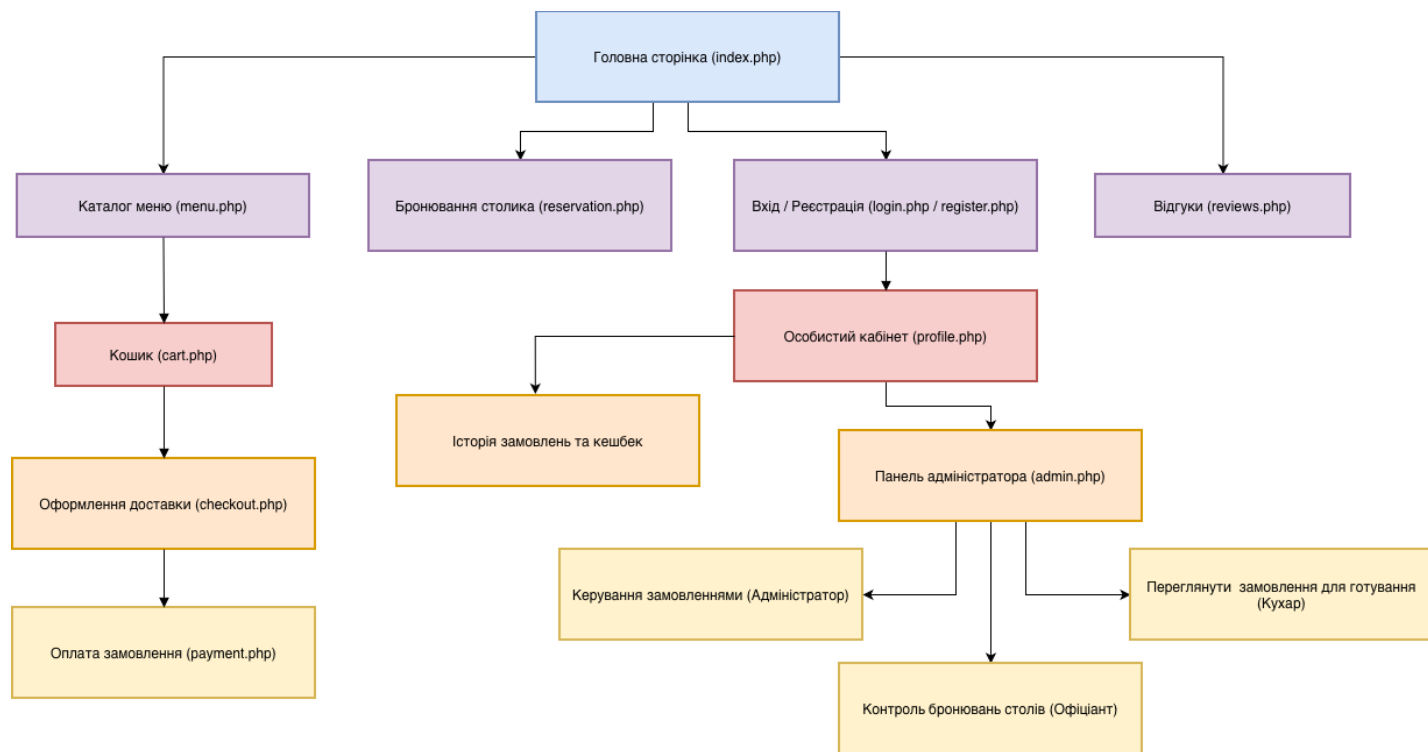


Рис. 3.1. Ієрархічна структура сторінок та модулів вебсайту
Джерело: розроблено автором у draw.io

3.3. Макети сторінок веб-сайту

Візуальний інтерфейс інформаційної системи ресторану розроблено з урахуванням сучасних принципів користувацького досвіду (UX/UI дизайну). Головними критеріями при проектуванні сторінок були адаптивність під мобільні пристрої, висока читабельність текстового контенту, інтуїтивно зрозуміла навігація та швидкий доступ до цільових дій: замовлення їжі та онлайн-бронювання столика.

Головна сторінка (index.php) виконує роль презентаційної картки закладу. На ній розміщено фірмовий логотип ресторану, інтерактивне навігаційне меню, інформаційні блоки про особливості кухні, а також динамічний слайдер із відгуками реальних клієнтів. Дизайн виконано у стриманій кольоровій гамі, що підкреслює специфіку закладу (рис. 3.2).

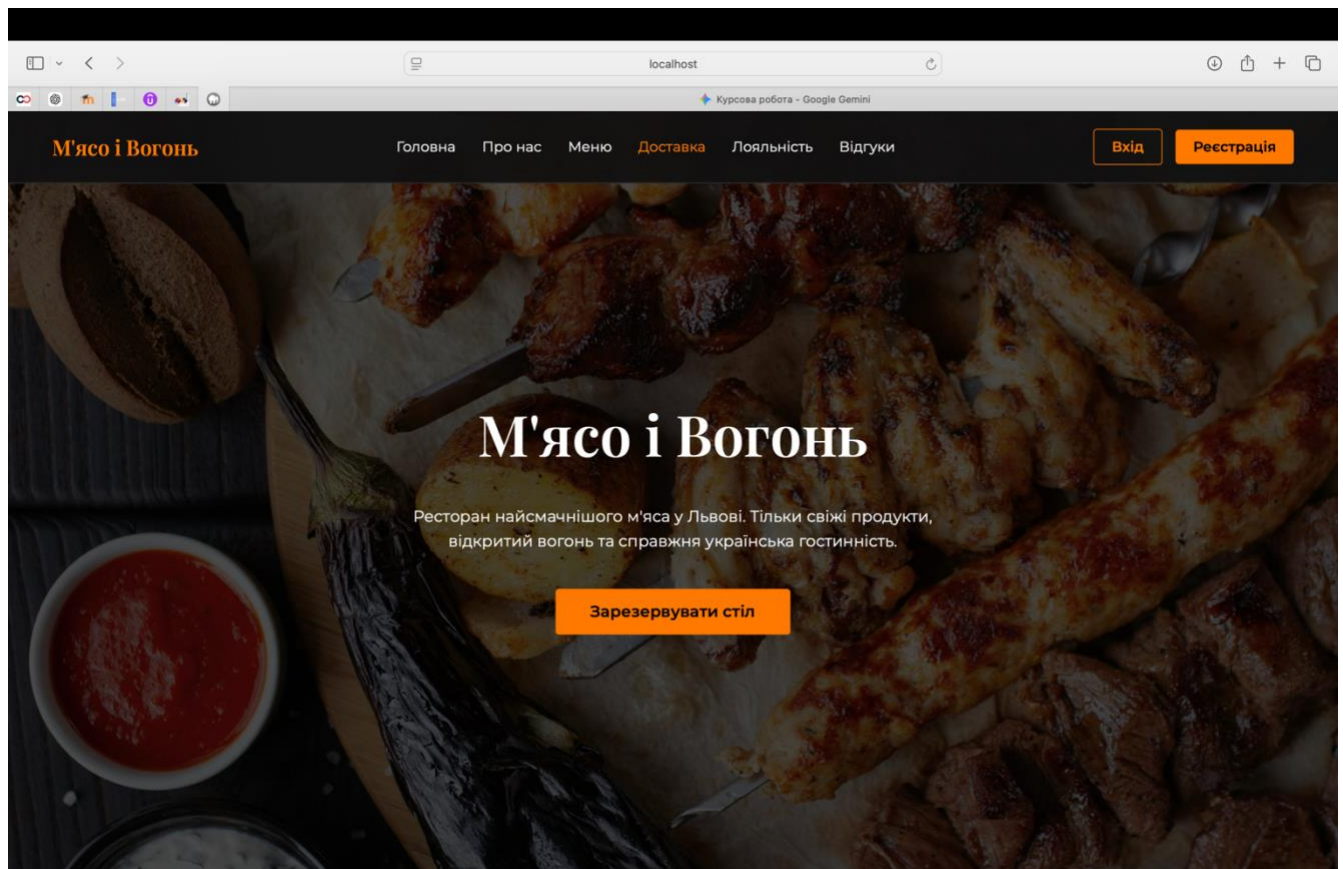


Рис. 3.2. Макет головної сторінки веб-сайту ресторану

Сторінка каталогу меню (menu.php) спроектована у вигляді зручної сітки карткових елементів. Кожна картка страви містить якісне зображення, її назву, точну вартість та кнопку інтерактивного додавання до кошика. У верхній частині сторінки реалізовано панель фільтрації, яка дозволяє користувачеві в один клік перемикатися між категоріями (Гарячі страви, Салати, Напої), мінімізуючи час на пошук потрібної позиції (рис. 3.3).

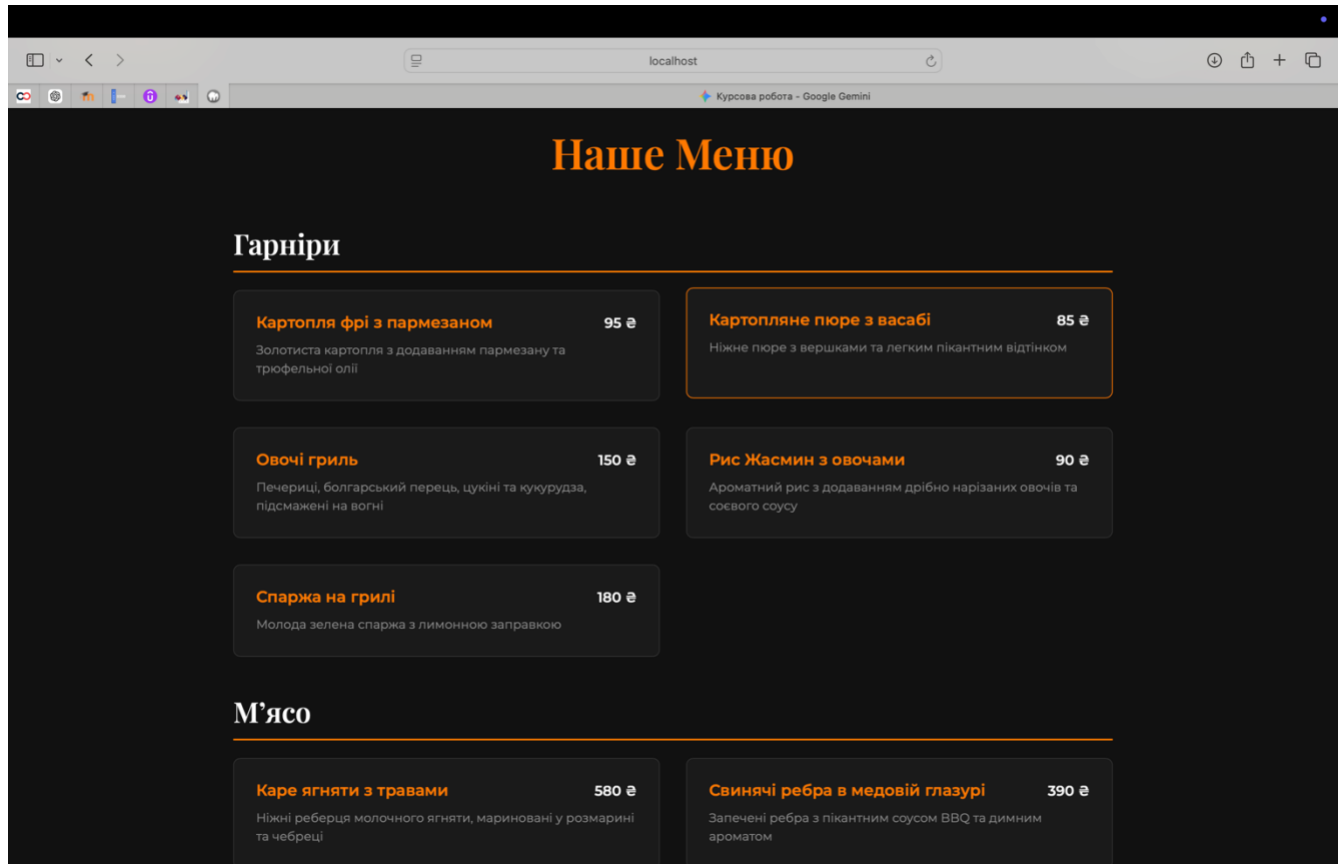


Рис. 3.3. Макет сторінки ресторанного меню

Інтерфейс модуля онлайн-бронювання (reservation.php) містить лаконічну форму з чітко визначеними полями введення. Для запобігання помилковому відправленню некоректних даних на стороні клієнта налаштовано обов'язкову валідацію: календар обмежує вибір минулих дат, а поле введення кількості гостей та номеру столика приймає лише позитивні цілі числа, що повністю відповідає доменним обмеженням нашої реляційної моделі (рис. 3.4).

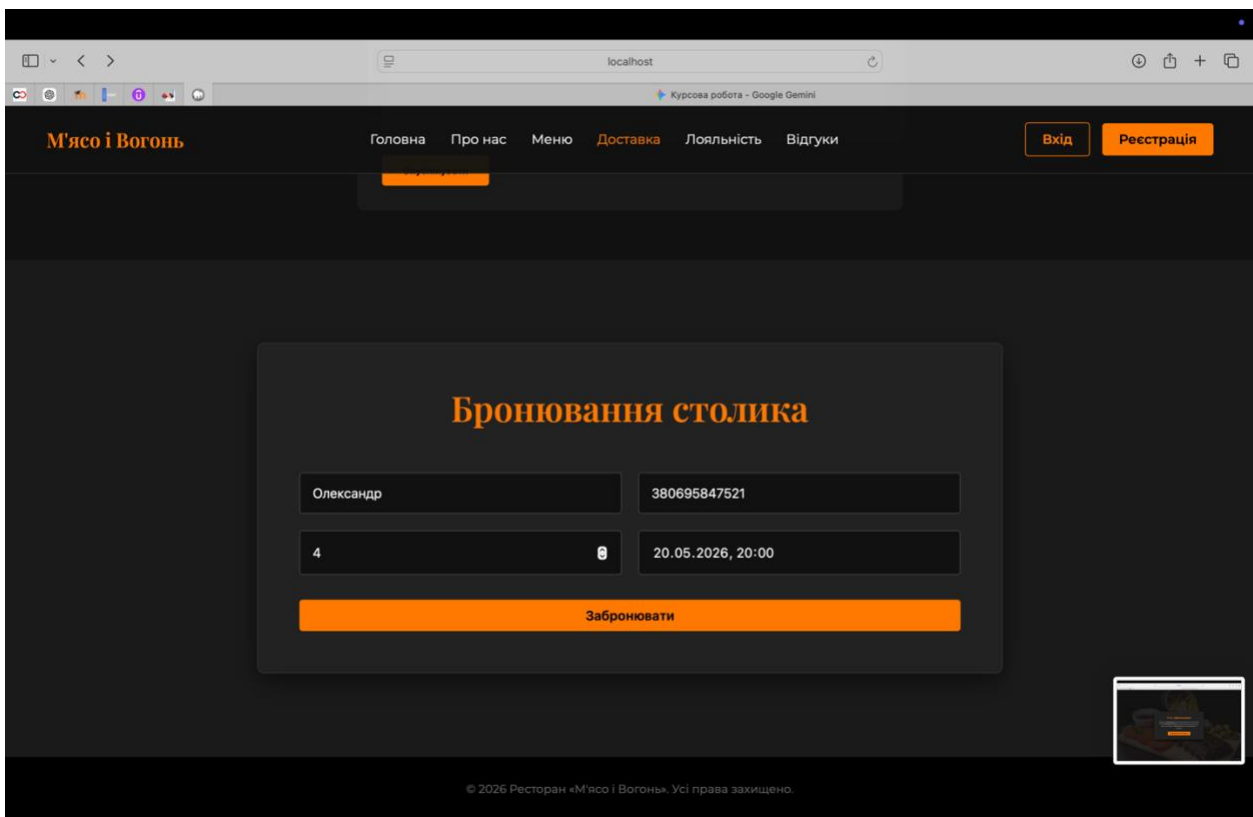


Рис. 3.4. Макет сторінки онлайн-бронювання столика

Інтерфейс адміністратора (admin.php), який є центром управління бізнес-процесами закладу. На відміну від клієнтської частини, робочий термінал розроблений для швидкого моніторингу ключових показників. На сторінці адміністратора відображається блок аналітичної звітності, де в реальному часі підсумовуються фінансові показники (доходи за місяцями) та рейтинг популярності страв. Також реалізовано модуль керування статусами меню, що дозволяє адміністратору миттєво змінювати доступність страв для замовлення (активувати або деактивувати позиції) залежно від наявності інгредієнтів на кухні (рис. 3.5). Таке візуальне рішення дозволяє мінімізувати час на оперативне керування закладом та запобігати замовленню відсутніх позицій клієнтами

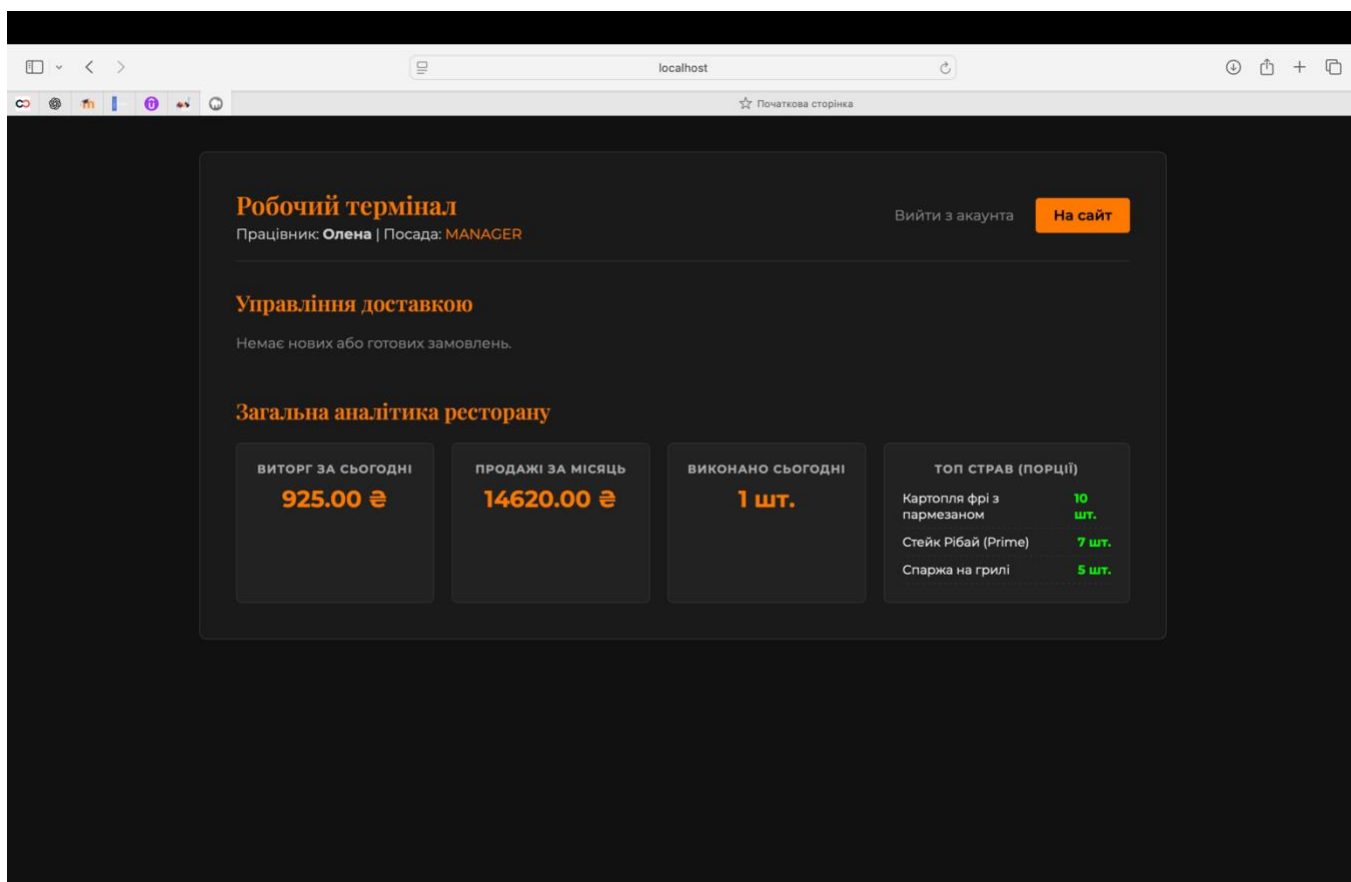


Рис. 3.5. Адміністративна панель із модулем аналітики та керуванням статусами меню

3.4. Програмування серверної частини

Серверна частина вебдодатка реалізована мовою програмування PHP версії 8.3. Головне завдання серверних скриптів - обробка HTTP-запитів від клієнта, виконання внутрішньої бізнес-логіки ресторану та безпечна взаємодія з базою даних MySQL. Усі процеси розробки, конфігурації та тестування серверного коду проводилися у локальному віртуальному середовищі MAMP (рис. 3.6).

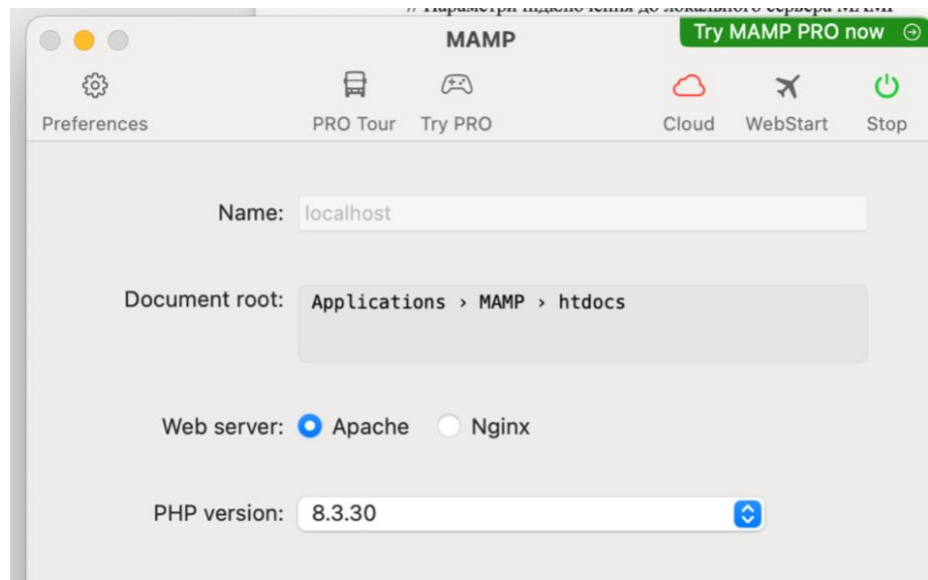


Рис. 3.6. Локальне серверне середовище MAMP із активними службами Apache та MySQL

Для підключення до бази даних `restaurant_db` використано об'єктно-орієнтований інтерфейс PDO (PHP Data Objects). Таке архітектурне рішення забезпечує надійний захист від SQL-ін'єкцій завдяки підтримці підготовлених запитів (prepared statements) та дозволяє гнучко обробляти винятки у разі технічних збоїв під час з'єднання. Фізичне розміщення структури таблиць на локальному сервері контролюється через СКБД MySQL (рис. 3.7).

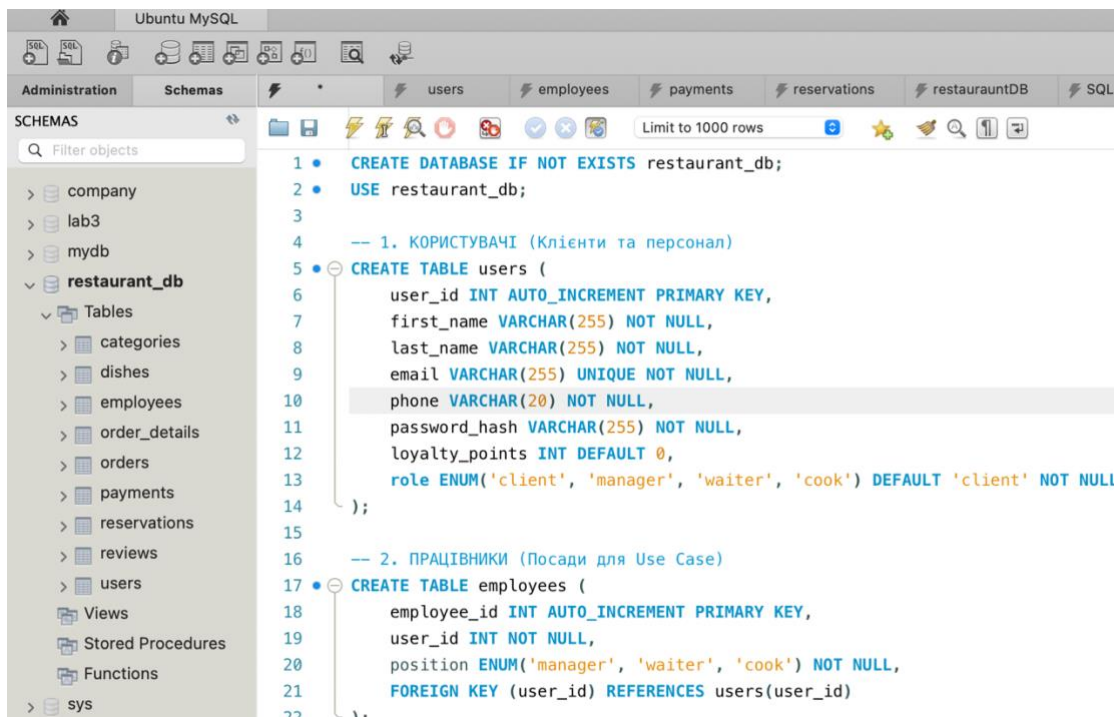


Рис. 3.7. Структура таблиць бази даних restaurant_db у СКБД MySQL

Процес формування контенту на сторінках відбувається динамічно. Наприклад, для відображення актуального каталогу на сторінці menu.php, серверний скрипт виконує реляційний SQL-запит до таблиць dishes та categories. Скрипт вибирає лише ті позиції, які мають прапорець наявності (is_available = 1), та передає їх у вигляді асоціативного масиву для подальшого виведення в HTML-розмітку (лістинг 3.1).

Лістинг 3.1. Підключення до бази даних та генерація меню

```
<?php

// Параметри підключення до локального сервера MAMP

$host = '127.0.0.1';

$db = 'restaurant_db';

$user = 'root';

$pass = 'root';
```

```

$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";

$options = [
    PDO::ATTR_ERRMODE          => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES => false,
];

try {
    // Ініціалізація з'єднання через PDO

    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (\PDOException $e) {
    throw new \PDOException($e->getMessage(), (int)$e->getCode());
}

// Вибірка страв та їхніх категорій

$sql = "SELECT d.dish_name, d.price, d.description, c.category_name
        FROM dishes d
        JOIN categories c ON d.category_id = c.category_id
        WHERE d.is_available = 1";

$stmt = $pdo->query($sql);

```

```
// Цикл для виведення карток страв на сайті

while ($row = $stmt->fetch()) {

    echo "<div class='menu-item'>";

    echo "<h3>" . htmlspecialchars($row['dish_name']) . "</h3>";

    echo "<p class='price'>" . htmlspecialchars($row['price']) . " €</p>";

    // Перевірка на наявність опису перед виведенням

    echo "<p>" . htmlspecialchars($row['description'] ?? "") . "</p>";

    echo "</div>";

}

?>
```

Для безпечного виведення текстових даних із бази (таких як опис страви або ім'я користувача) застосовується вбудована функція `htmlspecialchars()`. Вона перетворює спеціальні символи на HTML-сутності, унеможливорюючи виконання шкідливого коду (XSS-атаки) у браузері клієнта. Використання оператора об'єднання з `null` (`?? ""`) додатково захищає скрипт від помилок типу "Deprecated" у випадках, коли певні необов'язкові поля в базі залишаються порожніми.

Обробка вхідних даних із форм (наприклад, під час реєстрації або бронювання столика) супроводжується обов'язковою сервеною валідацією. Дані проходять перевірку на відповідність типам, після чого безпечно передаються у базу за допомогою методів `prepare()` та `execute()` об'єкта PDO, гарантуючи ізоляцію SQL-команд від користувацького вводу.

3.5. Програмування клієнтської частини

Клієнтська частина (frontend) вебдодатка ресторану «М'ясо і Вогонь» розроблена з використанням класичного стеку технологій: HTML5, CSS3 та JavaScript. Основною метою було створення інтуїтивно зрозумілого, адаптивного та візуально привабливого інтерфейсу, який коректно відображається як на десктопних, так і на мобільних пристроях.

Семантична розмітка HTML5 забезпечує правильну структуру сторінок, використання тегів <header>, <main>, <section> та <footer> покращує доступність сайту та його індексацію пошуковими системами. Візуальний стиль (CSS3) підкреслює концепцію стейкхаусу: використано лаконічну темну кольорову гаму з контрастними помаранчевими акцентами, що стилістично відсилає до тематики вогню та грилю. Для організації контенту, зокрема сітки меню ресторану, застосовано технології Flexbox та CSS Grid, що дозволило ефективно та рівномірно розподілити картки страв по екрану незалежно від його розміру.

Інтерактивність інтерфейсу забезпечується за допомогою JavaScript. Клієнтські скрипти відповідають за базову анімацію, роботу модальних вікон, а також за валідацію форм на стороні браузера. Зокрема, під час заповнення форми бронювання столика, JavaScript миттєво перевіряє коректність введених дат та контактних даних ще до відправки запиту на сервер. Це значно знижує навантаження на базу даних та покращує користувацький досвід, адже клієнт одразу бачить повідомлення про помилку у разі некоректного вводу (лістинг 3.2).

Лістинг 3.2. Клієнтська валідація форми бронювання за допомогою
JavaScript

```
<script>
document.addEventListener("DOMContentLoaded", function() {
    const reservationDateInput = document.getElementById("res_date");
```

```
const guestsInput = document.getElementById("guests_count");
const form = document.getElementById("reservation_form");

form.addEventListener("submit", function(event) {
    const selectedDate = new Date(reservationDateInput.value);
    const today = new Date();
    today.setHours(0,0,0,0);

    // Перевірка, щоб дата бронювання не була в минулому
    if (selectedDate < today) {
        alert("Помилка: Дата бронювання не може бути в минулому часі!");
        event.preventDefault(); // Зупиняємо відправку форми
        return;
    }

    // Перевірка на валідну кількість гостей
    if (guestsInput.value < 1 || guestsInput.value > 20) {
        alert("Помилка: Кількість гостей має бути від 1 до 20 осіб.");
        event.preventDefault();
    }
});
});
</script>
```

Використання JavaScript дозволило зробити вебдодаток більш динамічним, надаючи користувачам швидкий зворотний зв'язок та запобігаючи потраплянню некоректних або неповних даних до серверної частини системи.

3.6. Розміщення веб-сайту на локальному віртуальному середовищі або в Інтернеті

Для забезпечення функціонування інформаційної системи ресторану «М'ясо і Вогонь» та демонстрації її можливостей було обрано розгортання на локальному віртуальному середовищі на базі програмного комплексу MAMP під управлінням операційної системи macOS. Такий підхід дозволяє повністю симулювати роботу віддаленого продуктивного вебсервера, здійснювати безперебійне тестування серверної логіки та забезпечувати високу швидкість обробки запитів без необхідності закупівлі комерційного хостингу на етапі розробки та налагодження додатка.

Процес розміщення та конфігурації вебдодатка складався з кількох послідовних технічних етапів:

1. Конфігурація вебсервера, а саме, усі програмні файли сайту (скрипти PHP, стилі CSS, клієнтські скрипти JavaScript та медіафайли) було розміщено у кореневій директорії локального сервера за шляхом /Applications/MAMP/htdocs/. Головна сторінка закладу доступна через файл fuulfirsttry.php. Конфігурація вебсервера Apache дозволяє коректно обробляти внутрішні посилання, сесії користувачів та маршрутизацію запитів у межах локального домену localhost.

2. Міграція та налаштування бази даних. Для перенесення реляційної моделі даних у локальне середовище СКБД MySQL було використано інструмент MySQL Workbench. Через з'єднання з локальним сервером за адресою 127.0.0.1 виконано експорт розробленого SQL-скрипту створення структури таблиць restaurant_db. Після успішного імпорту структури було проведено фінальне заповнення та перевірку таблиць categories, dishes та users для забезпечення наявності демонстраційного контенту під час роботи з інтерфейсом.

3.Налаштування мережевої взаємодії. Зв'язок між серверними PHP-скриптами та СКБД MySQL реалізовано через стандартний мережевий порт 3306. Завдяки інтеграції з локальним оточенням, конфігураційні параметри PDO в PHP використовують облікові дані адміністратора (користувач root та пароль root), що гарантує стабільний доступ до таблиць. У випадках тестування системи через ізольовані віртуальні машини, наприклад у середовищі Parallels Desktop, доступ до вебсервера та бази даних організовано через виділену внутрішню IP-адресу 10.211.55.4, що забезпечує належний рівень безпеки та ізоляції даних від зовнішніх мереж.

Фінальна перевірка розгорнутої системи підтвердила коректність роботи всіх модулів: каталог страв динамічно підтягує актуальні ціни, категорії та описи з таблиць, реєстрація нових користувачів стабільно заносить записи до таблиці users, а форми онлайн-бронювання та замовлень безперебійно взаємодіють із відповідними реляційними відношеннями. Локальне середовище повністю підготовлене до фінальної презентації проєкту та його подальшого перенесення на продуктивний хмарний хостинг в Інтернеті.

3.7. Тестування взаємодії вебдодатка з базою даних

Для підтвердження коректності обміну даними між клієнтським інтерфейсом та базою даних проведено наскрізне тестування функціонала. Перевірка охоплює повний цикл обробки інформації: від введення даних користувачем у браузері до їх фізичного збереження у відповідних таблицях СКБД MySQL.

Тестування виконано на прикладі модуля онлайн-бронювання столика. Процес розпочинається на стороні клієнта, де користувач заповнює HTML-форму, вказуючи свої контактні дані та деталі замовлення (рис. 3.8).

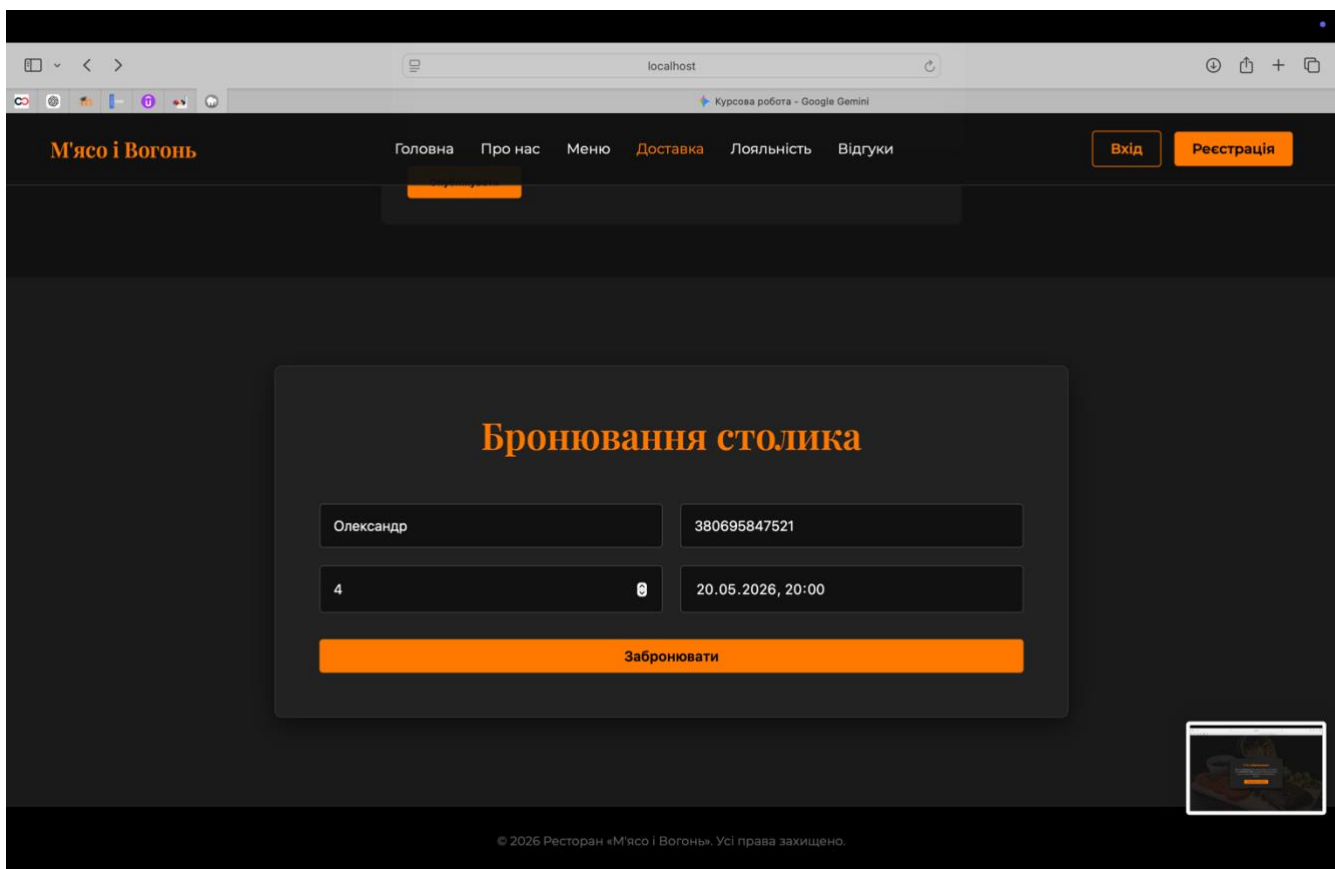
The image is a screenshot of a web browser displaying the 'М'ясо і Вогонь' website. The browser's address bar shows 'localhost'. The website has a dark theme. At the top, there is a navigation bar with links: 'Головна', 'Про нас', 'Меню', 'Доставка', 'Лояльність', and 'Відгуки'. To the right of these links are two buttons: 'Вхід' (Login) and 'Регістрація' (Registration). Below the navigation bar, there is a large, dark gray rectangular box with the title 'Бронювання столика' (Table Reservation) in orange. Inside this box, there are four input fields arranged in a 2x2 grid. The first field contains the text 'Олександр'. The second field contains the text '380695847521'. The third field contains the text '4'. The fourth field contains the text '20.05.2026, 20:00'. Below these fields is a large orange button with the text 'Забронювати' (Reserve). In the bottom right corner of the website, there is a small thumbnail image of a restaurant interior. At the very bottom of the page, there is a small copyright notice: '© 2026 Ресторан «М'ясо і Вогонь». Усі права захищено.'

Рис. 3.8. Введення даних у форму онлайн-бронювання на сайті

Після натискання кнопки підтвердження форми браузер ініціює POST-запит до сервера. Серверний PHP-скрипт отримує дані, проводить їх валідацію та формує параметризований SQL-запит INSERT INTO через інтерфейс PDO. Важливим елементом тестування є перевірка реакції інтерфейсу на успішну транзакцію. Як

видно на рис. 3.9, після успішного запису даних у базу, система миттєво відображає користувачеві модальне вікно з повідомленням про успішне бронювання. Це підтверджує коректну роботу логіки серверних скриптів та забезпечує позитивний користувацький досвід.

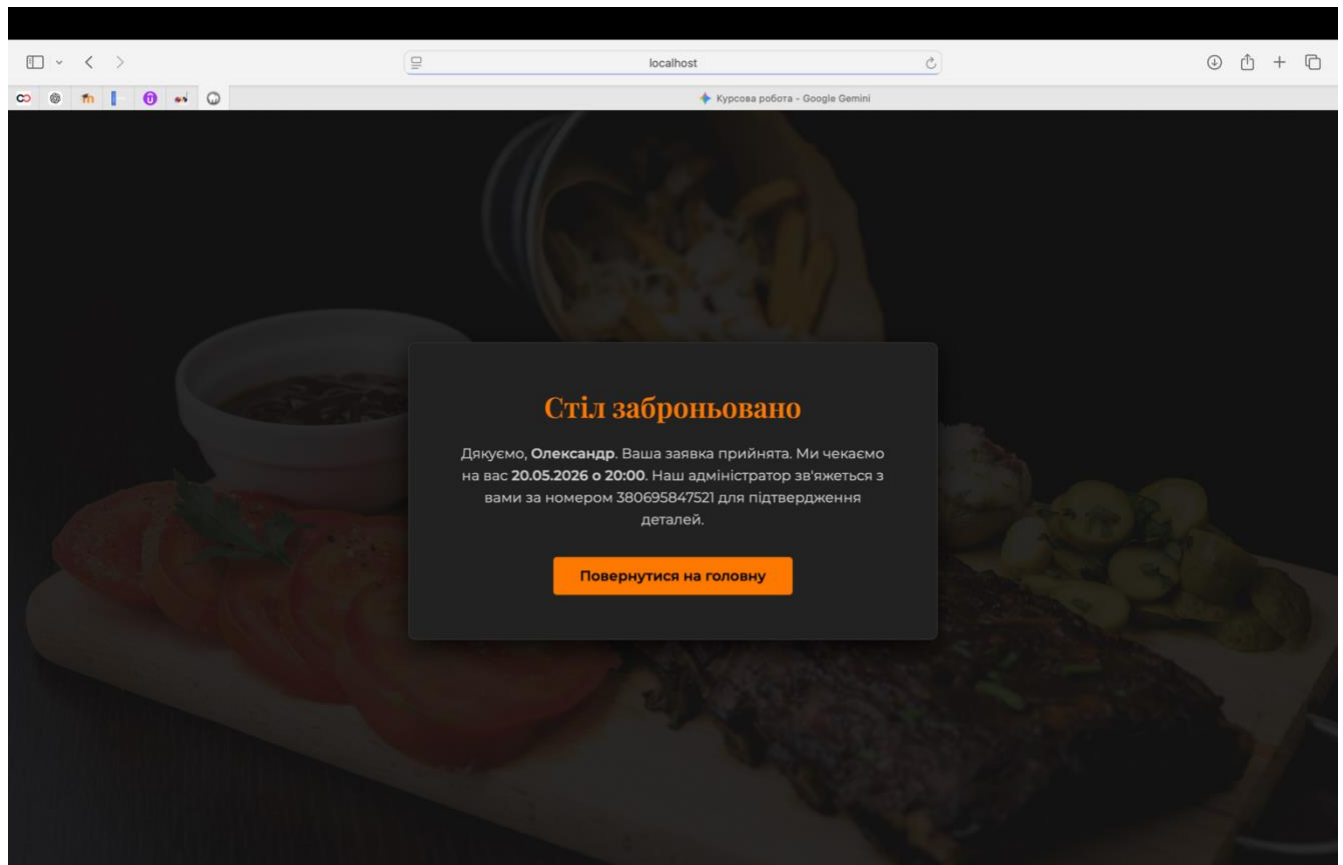


Рис. 3.9. Візуальне підтвердження успішного бронювання на стороні клієнта

Для фінальної верифікації успішності транзакції виконано контрольний запит до бази даних у середовищі MySQL Workbench. Результати вибірки з таблиці `reservations` підтверджують, що новий запис, який відповідає введеним на сайті даним (ім'я, телефон, дата), був фізично створений у базі даних (рис. 4).

The screenshot shows a database management interface. On the left, a sidebar lists the 'restaurant_db' schema and its tables: company, lab3, mydb, reservations, categories, dishes, employees, order_details, orders, payments, reviews, users, and Views. The main window displays a query: `SELECT * FROM restaurant_db.reservations;`. Below the query, a 'Result Grid' shows the following data:

reservation_id	user_id	guest_name	guest_phone	reservation_date	guests_cou...	table_number
1	HULL	Таня	380676068565	2026-01-15 16:30:00	4	HULL
2	HULL	Андрій Бойко	380676068565	2026-05-15 15:40:00	4	HULL
3	HULL	Степан	380976457832	2026-05-16 17:30:00	4	HULL
4	HULL	Степан	380976457832	2026-05-16 17:30:00	4	HULL
5	HULL	Богдан	380584683944	2026-05-15 17:30:00	2	HULL
6	HULL	Катерина	380685647318	2026-05-22 11:00:00	3	HULL
7	HULL	Петро	380694637483	2026-05-18 15:15:00	1	HULL
8	HULL	Євген	380685947328	2026-05-16 21:20:00	5	HULL
9	HULL	Олександр	380695847521	2026-05-20 20:00:00	4	HULL
HULL	HULL	HULL	HULL	HULL	HULL	HULL

Рис. 4. Відображення нового запису в таблиці reservations СКБД MySQL

Успішне проходження трьохетапного тесту (введення - інтерфейсна реакція - збереження в БД) підтверджує, що архітектура інформаційної системи налаштована правильно. Двостороння інтеграція фронтенду та бекенду функціонує без збоїв, забезпечуючи надійну реєстрацію користувацьких дій та збереження бізнес-даних ресторану.

ВИСНОВОК

У курсовій роботі досліджено теоретичні та практичні аспекти проектування, розробки та впровадження інформаційної системи прийому замовлень та обслуговування клієнтів ресторанного комплексу на прикладі закладу «М'ясо і Вогонь». Створення цієї системи дозволило автоматизувати ключові операційні процеси, пов'язані з обробкою замовлень, онлайн-бронюванням столиків, взаємодією з відвідувачами та веденням аналітичного обліку діяльності підприємства.

Під час виконання першого етапу роботи було детально проаналізовано предметну область, визначено основні вимоги до системи та розроблено модель варіантів використання. Це дозволило чітко розмежувати ролі користувачів (неавторизованих гостей, постійних клієнтів та адміністративного персоналу закладу) і сформулювати логічну структуру майбутнього вебдодатка.

У межах розробки архітектури збереження даних спроектовано реляційну базу даних `restaurant_db` у середовищі MySQL Workbench. Структуру бази даних, яка складається з дев'яти взаємопов'язаних таблиць, було успішно нормалізовано до третьої нормальної форми (3НФ), що дозволило повністю усунути надмірність інформації та запобігти виникненню аномалій модифікації даних. Реалізовані обмеження цілісності (первинні та зовнішні ключі, унікальні індекси та автоматичне збільшення ідентифікаторів) гарантують консистентність даних під час виконання транзакцій. Практична працездатність бази даних підтверджена розробкою та успішним тестуванням складних SQL-запитів з використанням агрегатних функцій та операторів об'єднання для аналізу фінансових надходжень ресторану за місяцями, рейтингу популярності страв у меню та стану бонусних рахунків користувачів у програмі лояльності.

Програмна реалізація вебдодатка виконана на базі комбінованого технологічного стеку. Клієнтська частина (frontend) розроблена за допомогою мов

HTML5 і CSS3 з використанням адаптивної верстки, що забезпечує коректне відображення темного фірмового інтерфейсу на різних типах пристроїв. Скрипти мовою JavaScript реалізують інтерактивність сторінок, обробку модальних вікон підтвердження та первинну валідацію введених користувачем даних, зменшуючи кількість некоректних запитів до сервера. Серверна частина (backend) побудована на мові програмування PHP версії 8.3. Взаємодія з базою даних MySQL реалізована через безпечний інтерфейс PDO за допомогою підготовлених запитів, що забезпечує надійний захист системи від атак типу SQL-ін'єкцій.

Наскрізне тестування розгорнутого у локальному віртуальному середовищі MAMP вебдодатка продемонструвало повну синхронізацію фронтенду та бекенду. Процес введення даних користувачем через форму онлайн-бронювання столика завершується миттєвим внесенням запису до таблиці reservations, відображенням графічного вікна успішного підтвердження операції в браузері та коректним відображенням даних у СКБД.

Розроблена інформаційна система ресторану «М'ясо і Вогонь» повністю готова до практичного використання, оскільки вона оптимізує роботу персоналу, мінімізує вплив людського фактора під час реєстрації замовлень та підвищує якість сервісу. Перспективами подальшого розвитку проєкту є інтеграція з платіжними шлюзами для онлайн-оплати чеків, впровадження розширеного CRM-модуля для автоматичного керування робочими змінами працівників та створення мобільного додатка для клієнтів закладу.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Бутенко А. Технології автоматизації системи управління у закладах ресторанного господарства / А. Бутенко // Економіка та суспільство. - 2022.
URL: https://tourlib.net/statti_ukr/chkhalo.htm
2. Хіхловська К. В. Автоматизація бізнес-процесів закладу ресторанного господарства : кваліфікаційна робота бакалавра / К. В. Хіхловська. – Тернопіль : ТНТУ ім. І. Пулюя, 2024. URL: https://elartu.tntu.edu.ua/bitstream/lib/45737/1/dyplom_%20Khikhlovskaya%20K_V_2024.pdf
3. Дзюба Н. Громадське харчування: від документообігу до обліку / Н. Дзюба // Податки та бухгалтерський облік. URL: <https://i.factor.ua/ukr/journals/nibu/golden/g-03/article-3585.html?srsId=AfmBOoprUYnfU0x0QJzXejeCddbO72toaoCyr47uEMJupgoRwQw5BceQ>
4. Що таке бекенд.
URL: <https://pinta.com.ua/uk/blog/backend-web-development-explained/>
5. Що таке MySQL і як він зазвичай використовується у веб-розробці?
URL: <https://uk.eitca.org/web-development/eitc-wd-pmsf-php-and-mysql-fundamentals/getting-started-with-mysql/introduction-to-mysql/examination-review-introduction-to-mysql/what-is-mysql-and-how-is-it-commonly-used-in-web-development/>
6. Пасічник В. В. Організація баз даних та знань / В. В. Пасічник, В. А. Резніченко. – К.: Видавнича група BVH, 2006. – с.48-67
URL: https://duikt.edu.ua/uploads/1_1753_28577101.pdf

7. Офіційна документація PHP: Hypertext Preprocessor. URL: <https://www.php.net/docs.php>
8. Офіційна документація MySQL 8.0 Reference Manual. URL: <https://dev.mysql.com/doc/refman/8.0/en/>
9. Веллінг Л., Томсон Л. Розробка веб-додатків за допомогою PHP і MySQL. 5-те вид. Київ: Діалектика, 2023. 880 с.
10. Документація з об'єктно-орієнтованого інтерфейсу PDO (PHP Data Objects). URL: <https://www.php.net/manual/en/book.pdo.php>
11. Нормалізація баз даних та проєктування реляційних моделей. URL: <https://www.techopedia.com/definition/1234/database-normalization> (дата звернення: 13.05.2026).
12. Специфікація HTML5 від W3C. URL: <https://html.spec.whatwg.org/> (дата звернення: 14.05.2026).

ДОДАТОК А

Код бази даних restaraut_db:

```
CREATE DATABASE IF NOT EXISTS restaurant_db;
USE restaurant_db;

-- 1. Клієнти
CREATE TABLE users (
    user_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(20) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    loyalty_points INT DEFAULT 0,
    city VARCHAR(100),
    street VARCHAR(255),
    house_number VARCHAR(10),
    apartment_number VARCHAR(10)
);

-- 2. Персонал
CREATE TABLE employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(255) NOT NULL,
    last_name VARCHAR(255) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    phone VARCHAR(20) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    position ENUM('manager', 'waiter', 'cook') NOT NULL
```


);

-- 3. Категорії та Страви

```
CREATE TABLE categories (  
    category_id INT AUTO_INCREMENT PRIMARY KEY,  
    category_name VARCHAR(255) NOT NULL,  
    is_active TINYINT(1) DEFAULT 1 NOT NULL  
);
```

```
CREATE TABLE dishes (  
    dish_id INT AUTO_INCREMENT PRIMARY KEY,  
    category_id INT NOT NULL,  
    dish_name VARCHAR(255) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    is_available TINYINT(1) DEFAULT 1 NOT NULL,  
    is_recommended TINYINT(1) DEFAULT 0,  
    FOREIGN KEY (category_id) REFERENCES categories(category_id)  
);
```

-- 4. Заовлення та склад заовлень

```
CREATE TABLE orders (  
    order_id INT AUTO_INCREMENT PRIMARY KEY,  
    employee_id INT NULL,  
    order_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    total_price DECIMAL(10,2) NOT NULL,  
    status VARCHAR(20) DEFAULT 'new',  
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id)  
);
```

```
CREATE TABLE order_details (  

```

```
detail_id INT AUTO_INCREMENT PRIMARY KEY,  
order_id INT NOT NULL,  
dish_id INT NOT NULL,  
quantity INT NOT NULL,  
unit_price DECIMAL(10,2) NOT NULL,  
FOREIGN KEY (order_id) REFERENCES orders(order_id),  
FOREIGN KEY (dish_id) REFERENCES dishes(dish_id)  
);
```

-- 5. Оплати (зв'язок з клієнтом)

```
CREATE TABLE payments (  
    payment_id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT UNIQUE NOT NULL,  
    user_id INT NOT NULL,  
    amount DECIMAL(10,2) NOT NULL,  
    payment_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
    city VARCHAR(100),  
    street VARCHAR(255),  
    house_number VARCHAR(10),  
    apartment_number VARCHAR(10),  
    FOREIGN KEY (order_id) REFERENCES orders(order_id),  
    FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

-- 6. Бронювання та Відгуки

```
CREATE TABLE reservations (  
    reservation_id INT AUTO_INCREMENT PRIMARY KEY,  
    user_id INT DEFAULT NULL,  
    guest_name VARCHAR(255),  
    guest_phone VARCHAR(20),
```

```
reservation_date DATETIME NOT NULL,  
guests_count INT NOT NULL,  
table_number INT,  
status ENUM('pending', 'confirmed', 'cancelled') DEFAULT 'pending',  
FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

```
CREATE TABLE reviews (  
review_id INT AUTO_INCREMENT PRIMARY KEY,  
user_id INT NULL,  
author_name VARCHAR(255) DEFAULT 'AhoHim',  
review_text TEXT NOT NULL,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
FOREIGN KEY (user_id) REFERENCES users(user_id)  
);
```

ДОДАТОК Б

Лістинг аналітичних SQL-запитів до бази даних

Запит 1. Отримання зведеної інформації про клієнтів, їхні замовлення та бонусний баланс (використання JOIN)

```
USE restaurant_db;

SELECT    u.first_name,    u.last_name,
u.phone,    u.loyalty_points,    o.order_date,
o.total_price, o.status

FROM users u

JOIN payments p ON u.user_id = p.user_id

JOIN orders o ON p.order_id = o.order_id;
```

Запит 2. Обчислення фінансових надходжень ресторану за місяцями (використання агрегатних функцій та групування)

```
USE restaurant_db;

SELECT  EXTRACT(MONTH FROM
p.payment_date) AS month, SUM(p.amount) AS
total_income

FROM payments p

JOIN orders o ON p.order_id = o.order_id

WHERE o.status = 'completed'

GROUP BY EXTRACT(MONTH FROM
p.payment_date)

ORDER BY month;
```

Запит 3. Формування рейтингу популярності страв у меню (від найбільш до найменш затребуваних)

```
USE restaurant_db;
```

```
SELECT d.dish_name, SUM(od.quantity)
AS total_sold

FROM dishes d

JOIN order_details od ON d.dish_id =
od.dish_id

GROUP BY d.dish_name

ORDER BY total_sold DESC;
```